Software Requirements and Architecture Document

Process Change Impact Simulator

Jagruthi Ampally

FA26 – COMP-699-A – PROFESSIONAL SEMINAR

Professor: David Pitts

Rivier University

1. Introduction

1.1 Project Overview

I plan to develop a Python application called Process Change Impact Simulator. My project will help organizations understand the effects of changing an activity, approval, condition, or rule in an existing business process.

Business processes contain connected activities. For example, a purchase approval process may include submitting a request, checking the purchase amount, obtaining manager approval, obtaining finance approval, and placing an order. If I change an approval rule, several later activities may also be affected. Identifying these effects manually can take time and may lead to missed dependencies.

I will represent business processes as dependency graphs, where activities and decisions are nodes and relationships are edges. When a user changes one component, my application will identify the activities that depend directly or indirectly on it. I will also simulate the original and modified processes to compare execution paths, skipped activities, required steps, and final outputs.

1.2 Problem Statement and Objectives

I identified the need for a simple way to understand the possible effects of proposed business process changes. My application will address this problem by combining graph analysis and simulation.

My main objectives are to:

- Represent business processes and their dependencies.
- Allow users to create, edit, import, and export process definitions.

- Identify direct and indirect downstream dependencies.
- Support activity, approval, and rule-based dependencies.
- Simulate original and modified processes.
- Compare affected activities, execution paths, and final outputs.
- Display results through a simple interface and clear visualizations.
- Test the application using synthetic business process data.

1.3 Project Scope

I will focus on processes containing activities, decisions, conditions, dependencies, inputs, and outputs. I will use employee onboarding and purchase approval as sample workflows. The application will be a prototype for reviewing proposed changes and will not modify live organizational workflows or perform actual approvals.

2. System Requirements

I have divided my requirements into functional requirements, which describe what the application will do, and non-functional requirements, which describe its expected quality and performance.

2.1 Functional Requirements (40)

FR-01: I will provide an option to create a new business process.

FR-02: I will allow users to enter a process name and description.

FR-03: I will allow users to add activities to a process.

FR-04: I will support tasks, approvals, decisions, and output activities.

FR-05: I will allow users to define conditions for activities.

FR-06: I will allow users to connect activities through directed dependencies.

FR-07: I will support direct activity dependencies.

FR-08: I will support approval-based dependencies.

FR-09: I will support rule-based dependencies.

FR-10: I will allow users to define process inputs and outputs.

FR-11: I will allow users to import process definitions from JSON files.

FR-12: I will allow users to export process definitions to JSON files.

FR-13: I will validate imported process data.

FR-14: I will check unique activity identifiers and required fields.

FR-15: I will identify invalid activity references and missing connections.

FR-16: I will detect circular dependencies.

FR-17: I will allow users to select a component for impact analysis.

FR-18: I will allow users to define a proposed change.

FR-19: I will identify direct downstream dependencies.

FR-20: I will identify indirect downstream dependencies using graph traversal.

FR-21: I will identify affected approvals and decision points.

FR-22: I will classify affected components by dependency type.

FR-23: I will distinguish direct and indirect impacts.

FR-24: I will display downstream activities in a list.

FR-25: I will display the process dependency graph.

FR-26: I will highlight affected activities in the graph.

FR-27: I will simulate the original process.

FR-28: I will simulate the modified process.

FR-29: I will evaluate decision conditions using supplied inputs.

FR-30: I will record activities executed during simulation.

FR-31: I will identify activities skipped because their conditions are not met.

FR-32: I will identify activities that become required after a change.

FR-33: I will compare the execution paths of both process versions.

FR-34: I will compare the final outputs of both versions.

FR-35: I will summarize differences between the original and modified processes.

FR-36: I will allow users to view affected activity details.

FR-37: I will display results using graphs and visualizations.

FR-38: I will allow users to export impact analysis results.

FR-39: I will display clear error messages when analysis or simulation fails.

FR-40: I will provide sample datasets and test scenarios for demonstration.

2.2 Non-Functional Requirements (20)

NFR-01: I will design an interface that is easy to understand.

NFR-02: I will use clear labels and simple instructions.

NFR-03: I will organize the code into separate modules.

NFR-04: I will use meaningful variable and function names.

NFR-05: I will handle invalid inputs without crashing during normal use.

NFR-06: I will validate process data before analysis.

NFR-07: I will ensure consistent results for identical inputs.

NFR-08: I will support branching and merging dependency structures.

NFR-09: I will target an analysis time of five seconds or less for a graph with up to 1,000 nodes and 3,000 edges under normal conditions on my development computer.

NFR-10: I will design the application to work within available system resources.

NFR-11: I will display useful error messages.

NFR-12: I will use synthetic data for testing and demonstration.

NFR-13: I will avoid storing unnecessary sensitive business information.

NFR-14: I will separate data handling from interface code wherever practical.

NFR-15: I will use supported versions of Python and the selected libraries.

NFR-16: I will document the installation requirements.

NFR-17: I will clearly distinguish affected and unaffected activities.

NFR-18: I will support common desktop browsers compatible with Streamlit.

NFR-19: I will test sequential, branching, merging, and circular dependencies.

NFR-20: I will document system limitations and explain that results depend on the supplied process data and rules.

3. System Architecture

I will use a modular architecture with five main components.

- **User Interface:** I will use Streamlit to create forms, buttons, graphs, and result displays.
- **Process Manager:** I will manage process creation, editing, validation, import, and export.
- **Dependency Analyzer:** I will use NetworkX to represent activities as nodes and dependencies as directed edges. I will use graph traversal algorithms to identify downstream activities and detect cycles.
- **Simulation and Comparison Engine:** I will evaluate process conditions, record executed and skipped activities, and compare the original and modified versions.
- **Visualization Module:** I will use Plotly and Matplotlib to display dependency graphs and simulation results.

The main architecture flow will be:

Streamlit Interface → Process Manager and Validation → Dependency Graph → Impact Analysis and Simulation → Comparison Engine → Results and Visualizations

I will keep dependency analysis separate from simulation because an activity may depend on a changed component without being executed in every scenario. If a circular dependency prevents a valid execution order, the application will report the issue instead of running an invalid simulation.

4. Data Design

4.1 Data Model

I will use JSON to store and exchange process definitions. Each activity will have a unique identifier, name, type, and relevant conditions. Each dependency will identify its source,

destination, and relationship type. I will use NetworkX to convert the process definition into a graph and Pandas to organize process data and compare results.

The main data entities will be:

- **Process:** ID, name, description, starting activity, and final activities.
- **Activity:** ID, name, type, description, and conditions.
- **Dependency:** Source activity, destination activity, and dependency type.
- **Rule:** Condition, comparison operator, and expected value.
- **Input and Output:** Input values and final process results.
- **Change Record:** Selected component, original value, and proposed value.
- **Simulation Result:** Executed activities, skipped activities, paths, and outputs.

4.2 Sample Data and Validation

I will create synthetic employee onboarding and purchase approval workflows based on common organizational activities and publicly available workflow documentation. I will not use confidential organizational information.

For onboarding, I will include activities such as verifying employee information, creating an account, granting access, and completing onboarding. For purchase approval, I will include submitting a request, checking the amount, obtaining approvals, and placing an order.

I will test changes such as modifying an approval threshold, removing a condition, and changing a dependency. I will validate required fields, unique identifiers, dependency references, and rule formats before running the analysis. I will store process definitions in JSON and export results in CSV or JSON.

5. Technologies and Tools

I have selected the following technologies for my project:

- **Python:** I will develop the main application logic and simulation.
- **Streamlit:** I will build the interactive user interface.
- **NetworkX:** I will create dependency graphs, trace downstream activities, and detect cycles.
- **Pandas:** I will organize process data and compare simulation results.
- **NumPy:** I will use it for numerical calculations if required.
- **Plotly:** I will display interactive graphs and results.
- **Matplotlib:** I will create additional charts when needed.
- **JSON:** I will store and exchange process definitions.
- **Git:** I will track source code changes.
- **pytest:** I will test graph analysis, data validation, and simulation.

6. Risks and Limitations

I have identified the following risks and responses:

- **Incorrect process data:** I will validate data before analysis.
- **Complex dependencies:** I will test different graph structures and use suitable graph algorithms.
- **Circular dependencies:** I will detect cycles and report them.
- **Incorrect rule evaluation:** I will compare simulation results with predefined expected outcomes.

- **Large process models:** I will measure execution time and check resource usage.
- **Unclear results:** I will use clear labels, graph highlighting, and summary displays.
- **Limited development time:** I will complete the core features before optional improvements.
- **Incorrect workflow assumptions:** I will document the conditions and assumptions used in sample processes.

The application's accuracy will depend on the quality of the supplied data and rules. It will not automatically understand business rules that are missing from the process model. I will present its results as decision-support information rather than a guarantee that every possible effect has been identified.

7. Project Schedule

My course ends on December 8, 2026. I will use the period from October 14 to December 8, 2026, to complete the project.

- **October 14–20:** I will finalize requirements, project scope, and architecture.
- **October 21–27:** I will design the data model and prepare synthetic process datasets.
- **October 28–November 3:** I will implement process creation, JSON import/export, and data validation.
- **November 4–10:** I will implement the dependency graph and downstream impact analysis.
- **November 11–17:** I will develop simulation and process comparison features.
- **November 18–24:** I will build the Streamlit interface and visualizations.

- **November 25–December 1:** I will test the application, correct errors, and evaluate the results.
- **December 2–8:** I will complete documentation, organize the source code and datasets, and prepare the final demonstration.

8. Evaluation and Testing Plan

I will evaluate my application by comparing actual results with expected outcomes for each test case. I will use synthetic datasets so that I can control the process structure, conditions, and proposed changes.

8.1 Test Scenarios

I will test the following scenarios:

1. **Sequential process:** I will change an early activity and check whether the downstream activities are identified.
2. **Branching process:** I will change a decision condition and check whether the correct path is followed.
3. **Multiple approvals:** I will change an approval rule and check which approval activities are affected.
4. **Approval threshold:** I will change a purchase amount threshold and compare approval paths.
5. **Removed condition:** I will check whether an approval is skipped when its condition no longer applies.
6. **Multiple dependencies:** I will verify that all reachable downstream activities are identified.

7. **Circular dependency:** I will check whether the application detects and reports a cycle.
8. **Invalid data:** I will check how the application handles missing fields and invalid references.
9. **Output comparison:** I will check whether a rule change produces the expected output differences.
10. **Performance:** I will measure analysis and simulation times for different graph sizes.

8.2 Evaluation Criteria

I will evaluate the application based on dependency accuracy, simulation accuracy, change detection, data validation, performance, usability, and reliability. I will compare the results with expected outcomes and repeat tests with identical inputs to confirm consistency.

I will maintain test records containing the test case, input data, expected result, actual result, and pass or fail status. I will consider the project successful if it can create or import a valid process, identify downstream dependencies, simulate both versions, compare their results, and display the findings clearly.

9. Expected Outcome

I expect to develop a working prototype that demonstrates how graph algorithms can help users understand proposed business process changes. The application will identify downstream dependencies, simulate original and modified processes, and display changes in activities, execution paths, and final outputs.

Through this project, I will improve my practical knowledge of Python development, graph algorithms, data modeling, simulation, software architecture, and software testing.

References

- Object Management Group. (2013). *Business Process Model and Notation (BPMN), Version 2.0.2.* https://www.omg.org/spec/BPMN/2.0.2/
- NetworkX Developers. (n.d.). *NetworkX documentation.* https://networkx.org/documentation/stable/
- Streamlit. (n.d.). *Streamlit documentation.* https://docs.streamlit.io/
- Pandas Development Team. (n.d.). *pandas documentation.* https://pandas.pydata.org/docs/
- Plotly Technologies Inc. (n.d.). *Plotly Python graphing library.* https://plotly.com/python/
- Python Software Foundation. (n.d.). *Python documentation.* https://docs.python.org/3/
- pytest Development Team. (n.d.). *pytest documentation.* https://docs.pytest.org/
- IBM. (n.d.). *IBM documentation.* https://www.ibm.com/docs/
- Oracle. (n.d.). *Oracle documentation.* https://docs.oracle.com/